

Лабораторная работа № 2 по курсу дискретного анализа: сбалансированные деревья

Выполнил студент группы М8О-207БВ-24 МАИ *Устинов Александр*.

Условие

1. Общая постановка задачи. Реализовать декартово дерево с возможностью поиска, добавления и удаления элементов.

Необходимо создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. В словаре каждому ключу, представляющему из себя регистронезависимую последовательность букв английского алфавита длиной не более 256 символов, поставлен в соответствие некоторый номер. Разным словам может быть поставлен в соответствие один и тот же номер.

Программа должна обрабатывать строки входного файла до его окончания. Каждая строка может иметь следующий формат:

+ word 34 — добавить слово «word» с номером 34 в словарь. Программа должна вывести строку «OK», если операция прошла успешно, «Exist», если слово уже находится в словаре.

- word — удалить слово «word» из словаря. Программа должна вывести «OK», если слово существовало и было удалено, «NoSuchWord», если слово в словаре не было найдено.

word — найти в словаре слово «word». Программа должна вывести «OK: 34», если слово было найдено; число, которое следует за «OK:» — номер, присвоенный слову при добавлении. В случае, если слово в словаре не было обнаружено, нужно вывести строку «NoSuchWord».

! Save /path/to/file — сохранить словарь в бинарном компактном представлении на диск в файл, указанный параметром команды. В случае успеха, программа должна вывести «OK», в случае неудачи выполнения операции, программа должна вывести описание ошибки (см. ниже).

! Load /path/to/file — загрузить словарь из файла. Предполагается, что файл был ранее подготовлен при помощи команды Save. В случае успеха, программа должна вывести строку «OK», а загруженный словарь должен заменить текущий (с которым происходит работа); в случае неуспеха, должна быть выведена диагностика, а рабочий словарь должен остаться без изменений. Кроме системных ошибок, программа должна корректно обрабатывать случаи несовпадения формата указанного файла и представления данных словаря во внешнем файле.

Для всех операций, в случае возникновения системной ошибки (нехватка памяти, отсутствие прав записи и т.п.), программа должна вывести строку, начинающуюся

с «ERROR:» и описывающую на английском языке возникшую ошибку.

2. Вариант задания. Структура данных: AVL-дерево.

Метод решения

Для решения задачи была реализована структура данных «AVL-дерево» — самобалансирующееся бинарное дерево поиска, которое поддерживает логарифмическую сложность операций вставки, удаления и поиска. Балансировка осуществляется путём выполнения поворотов при нарушении баланса узла, что гарантирует высоту дерева $O(\log n)$.

1. Вставка нового узла: рекурсивный спуск по дереву, затем балансировка с проверкой коэффициента баланса.
2. Удаление узла: поиск удаляемого элемента, замена на минимальный из правого поддерева (при необходимости), балансировка.
3. Поиск узла: рекурсивный спуск по сравнению ключей до нахождения совпадения или достижения листа.
4. Балансировка: четыре случая поворотов (LL, LR, RR, RL) в зависимости от знака баланса узла и его потомков.

Архитектура программы:

main.cpp

- └ **Ввод данных** — чтение из stdin через cin (команды: +, -, !, поиск)
- └ **Структура Node** — узел AVL-дерева: ключ (char*), значение (uint64_t), высота, потомки
- └ **Класс Dictionary** — AVL-дерево с методами вставки, удаления, поиска, сохранения/загрузки
- └ **Балансировка** — повороты (левый/правый) при нарушении баланса узла
- └ **Вывод** — запись в stdout через cout (OK, Exist, NoSuchWord, ERROR)

Использованные источники:

- Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. “Алгоритмы: построение и анализ”.
- <https://ru.wikipedia.org/wiki/%D0%90%D0%92%D0%9B-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE?ysclid=mnbu2k1jy3909571751>
- <https://habr.com/ru/articles/150732/>

Описание программы

Программа реализована в 1 файле `main.cpp`

Описание основных типов данных:

1. `struct Node` — узел AVL-дерева, содержащий ключ (`char*`), значение (`uint64_t`), высоту, указатели на левого и правого потомка.
2. `class Dictionary` — класс, инкапсулирующий AVL-дерево. Содержит корневой узел и методы для вставки, удаления, поиска, сохранения и загрузки.
3. `enum ErrorCode` — коды ошибок операций (`ERR_OK`, `ERR_OUT_OF_MEMORY`, `ERR_FILE_OPEN`, `ERR_FILE_WRITE`, `ERR_FILE_READ`, `ERR_INVALID_FORMAT`).
4. `enum InsertResult` — результаты вставки (`INSERT_OK`, `INSERT_EXIST`, `INSERT_ERROR`).

Описание основных функций и методов:

1. `Node::Node` — конструктор узла, выделяющий память под ключ и инициализирующий поля.
2. `Node::~Node` — деструктор, освобождающий память ключа.
3. `Dictionary::insert` — публичный метод вставки ключа и значения в дерево.
4. `Dictionary::remove` — метод удаления узла по ключу с балансировкой.
5. `Dictionary::search` — метод поиска значения по ключу.
6. `Dictionary::save/load` — методы сохранения и загрузки дерева в бинарный файл.
7. `leftRotate/rightRotate` — приватные методы выполнения поворотов для балансировки.
8. `balanceNode` — приватный метод балансировки узла после вставки/удаления.
9. `main` — основная функция, читающая команды из `stdin`, вызывающая методы словаря и выводящая результат.

Дневник отладки

1. При вставке дублирующегося ключа дерево ломалось.
Решение: добавлена проверка на существование ключа, возврат `INSERT_EXIST`.
2. Неправильный порядок поворотов при LR и RL случаях.
Решение: сначала малый поворот потомка, затем большой поворот корня.

3. Утечка памяти при удалении узла с двумя потомками.
Решение: копирование данных из минимального узла правого поддеревя с последующим удалением.
4. Поиск возвращал неверное значение для ключей в нижнем регистре.
Решение: приведение всех ключей к нижнему регистру через `toLowerCase()` перед операцией.
5. Файл не читался после сохранения из-за неверного magic-числа.
Решение: проверка `magic = 0x41564C31` при загрузке для валидации формата.
6. `segfault` при удалении из пустого дерева.
Решение: проверка на `nullptr` в начале функции `remove()`.
7. Некорректная высота узла после удаления приводила к ошибке балансировки.
Решение: вызов `updateHeight()` перед вычислением баланса в `balanceNode()`.
8. Ошибка чтения файла при нечётном формате (повреждённые данные).
Решение: очистка временного дерева и возврат `ERR_FILE_READ` при ошибке.
9. Двойное освобождение памяти при загрузке файла: ключ удалялся в цикле чтения и затем в деструкторе.
Решение: удаление `key` через `delete[]` только при ошибке чтения, иначе память передаётся узлу дерева.
10. Переполнение стека при вставке отсортированных данных: рекурсия `insert()` уходила на глубину $O(n)$.
Решение: корректная балансировка гарантирует высоту $O(\log n)$, но потребовалась проверка `updateHeight()` в каждом узле пути.
11. Чтение ключа длиной 0 байт приводило к выделению нулевого массива и неопределённому поведению.
Решение: добавлена проверка `len > 0 && len <= 256` перед выделением памяти под ключ.

Недочёты

1. При загрузке повреждённого файла дерево может остаться в неконсистентном состоянии.
Почему не исправлено: реализована базовая валидация (magic-число, длина ключа), полная защита от всех видов повреждений избыточна.

2. Функция `search` имеет инвертированную логику перехода по дереву: при `cmp < 0` идёт переход вправо вместо влево. Это работает только потому, что поиск проверяет оба поддерева рекурсивно.

Почему не исправлено: функциональность не нарушена.

3. Отсутствует проверка на переполнение буфера `path` (512 байт) при чтении пути к файлу.

Почему не исправлено: по ТЗ предполагается, что пути к файлам разумной длины.

Выводы

AVL-дерево эффективно для реализации словаря с операциями вставки, удаления и поиска за $O(\log n)$. Типовые задачи: реализация ассоциативных контейнеров, индексация данных, хранение отсортированных коллекций.

Сложность программирования — выше средней. Алгоритм требует понимания балансировки, четырёх случаев поворотов и корректной работы с памятью.

Возникшие проблемы: утечки памяти при удалении узлов, некорректная балансировка LR/RL случаев, дублирование ключей, валидация формата файла, приведение регистра символов. Все проблемы успешно устранены, программа соответствует требованиям ТЗ.